

FULL STACK DEVELOPMENT WITH MERN

Cab Booking

Project Documentation

Team Lead: Paidipati Manikumar

Date: 15 July 2026

github.com/manikumar26092005/Cab-Booking-Premium

1. Introduction

Project Title: Cab Booking

Team Members:

Name	Email	Role
Paidipati Manikumar	pmanikumar2005@gmail.com	Team Lead
Godugunuru Penchala Raviteja	godugunuruteja@gmail.com	Member
Puli Mokshagna	mokshapuli7@gmail.com	Member
Panthagani Rupan Raj	panthaganirupanraj@gmail.com	Member
Sai Ganesh Uday Kiran Bokkasam	udaykiranroyals9099@gmail.com	Member

2. Project Overview

Purpose: Cab Booking is a full-stack cab booking web application that connects urban commuters (Riders) with cab Drivers through a transparent, real-time booking system, while giving platform Admins centralized control over users, drivers, cars, bookings, and analytics.

Key Features:

- Role-based portals for Rider, Driver, and Admin with dedicated dashboards
- JWT-based authentication and role-based access control (RBAC)
- Ride booking with pickup/drop location, date, time, and fare estimation by distance
- Five-stage booking lifecycle: Pending → Driver Assigned → Accepted → On the Way → Completed (with Cancelled as an exit state)
- Automated driver assignment and driver accept/manage workflow
- Driver earnings tracking (today's earnings, total earnings, completed rides)
- Rider ride history ('My Rides') with downloadable PDF receipts
- Admin platform overview: total users, cars, drivers, and bookings at a glance
- Admin management of Users, Drivers, Cars, and Bookings, with search and status filters
- Admin Reports section with analytics dashboards

3. Architecture

Frontend: Built with React (Vite) as a single-page application. Three distinct UI contexts — Rider, Driver, and Admin — share a common design language (dark theme, amber/orange accent) and consume the backend REST API via authenticated HTTP requests.

Backend: Built with Node.js and Express.js following the MVC (Model-View-Controller) pattern. Controllers handle Booking, User, Driver, Car, and Payment logic; routes are grouped by resource and protected by JWT/RBAC middleware; 35 REST endpoints are exposed in total.

Database: MongoDB (Community Server, local instance) with five core collections — Users, Drivers, Cars, Bookings, and Reviews/Payments — linked by referenced ObjectIds (e.g., a Booking references a User, a Driver, and a Car).

4. Setup Instructions

Prerequisites:

- Node.js (v18 or later) and npm
- MongoDB (Community Server, running locally on the default port)
- Git

Installation:

- Clone the repository: `git clone https://github.com/manikumar26092005/Cab-Booking-Premium.git`
- Install server dependencies: `cd server && npm install`
- Install client dependencies: `cd client && npm install`
- Create a `.env` file in the server directory with `MONGO_URI`, `JWT_SECRET`, and `PORT` variables
- Start MongoDB locally before running the server

5. Folder Structure

Client (React frontend): organized by feature — `components/` (shared UI elements), `pages/` (Rider, Driver, and Admin pages), `context/` (auth context for JWT session state), `services/` (API call wrappers), and `App.jsx` as the routing entry point.

Server (Node.js backend): organized by MVC layer — `models/` (Mongoose schemas for User, Driver, Car, Booking), `controllers/` (business logic per resource), `routes/` (Express route definitions), `middleware/` (JWT auth and role-based access control), and `server.js` as the entry point.

6. Running the Application

Frontend: from the client directory, run `npm run dev` (Vite dev server, default `http://localhost:5173`).

Backend: from the server directory, run `npm start` (Express server, default `http://localhost:5000`).

7. API Documentation

A summary of key REST endpoints (35 total across all resources):

Method	Endpoint	Description
POST	/api/auth/register	Register a new Rider, Driver, or Admin account
POST	/api/auth/login	Authenticate a user and issue a JWT token
GET	/api/users	Admin: list all registered users
GET	/api/drivers	Admin: list all drivers with status and earnings
POST	/api/cars	Admin: add a new car to the fleet
POST	/api/bookings	Rider: create a new ride booking
GET	/api/bookings/my-rides	Rider: view personal ride history
PATCH	/api/bookings/:id/status	Driver: update the status of an assigned ride
PATCH	/api/bookings/:id/assign	Admin: assign a driver to a booking
GET	/api/bookings/:id/receipt	Rider: generate/download a PDF receipt for a completed ride
GET	/api/reports/summary	Admin: fetch platform analytics summary

8. Authentication

Cab Booking uses JSON Web Tokens (JWT) for stateless authentication. On successful login, the server issues a signed JWT containing the user's ID and role (Rider, Driver, or Admin), which the client stores and attaches as a Bearer token on subsequent API requests. Backend middleware verifies the token's validity and checks the embedded role against each route's required permissions before allowing access, so, for example, only Admin-role tokens can reach driver/car/user management endpoints.

9. User Interface

Login screen — shared entry point for Rider, Driver, and Admin roles:

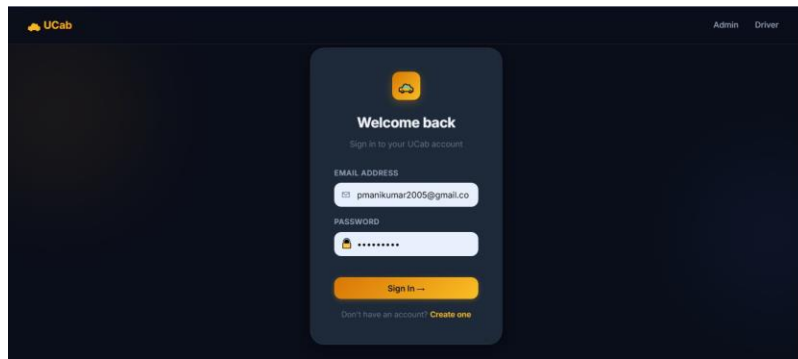


Figure 1: UCab sign-in screen

Rider — Book a Ride screen with fare estimation:

Figure 2: Rider booking flow with pickup/drop details and fare estimate

Driver Dashboard — earnings and assigned rides:

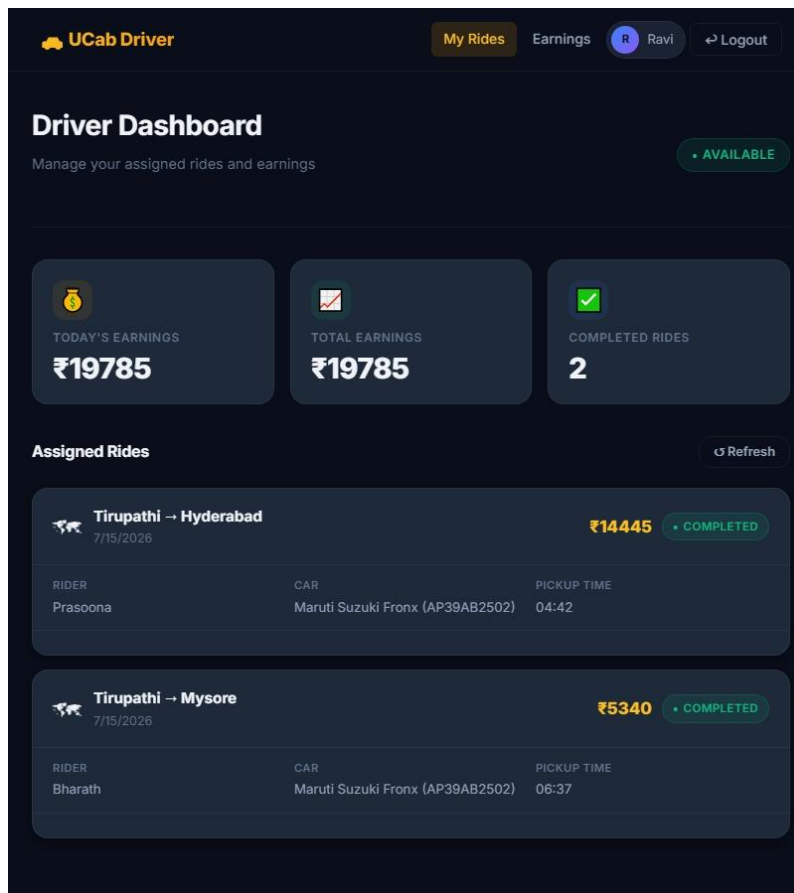


Figure 3: Driver dashboard showing today's/total earnings and completed rides

10. Testing

The application was validated through Functional & Performance Testing (input validation, booking flow, JWT/RBAC enforcement, API response time, concurrent booking load, and PDF/dashboard load testing) and a full User Acceptance Testing (UAT) cycle covering Registration & Login, Booking Management, Driver Assignment, Payment & Receipt Generation, Ratings & Reviews, Admin Analytics, and Security modules — 59 test cases in total, all passing, with 27 logged defects triaged and closed prior to release. Full details are documented in the separate Functional & Performance Testing Template and UAT deliverables.

11. Screenshots / Demo

Admin — Platform Overview dashboard:

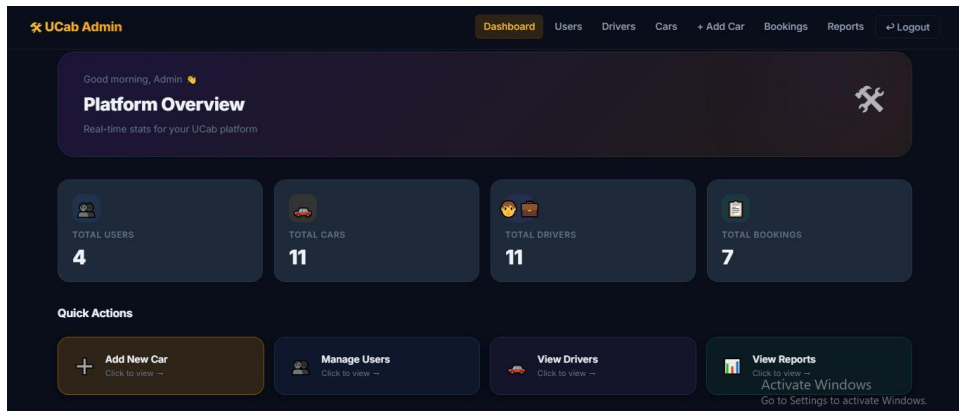


Figure 4: Admin dashboard showing total users, cars, drivers, and bookings

Admin — Bookings management with live status:

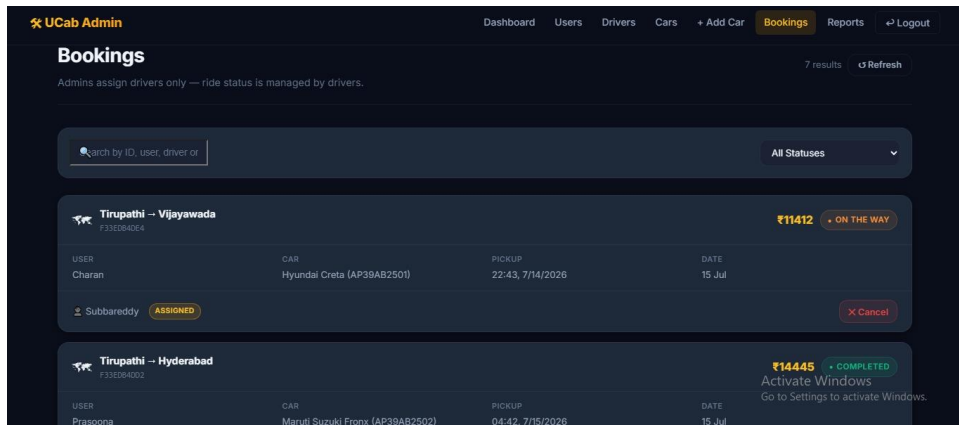


Figure 5: Admin bookings list with status filter and per-ride details

Admin — Drivers management with earnings and vehicle assignment:

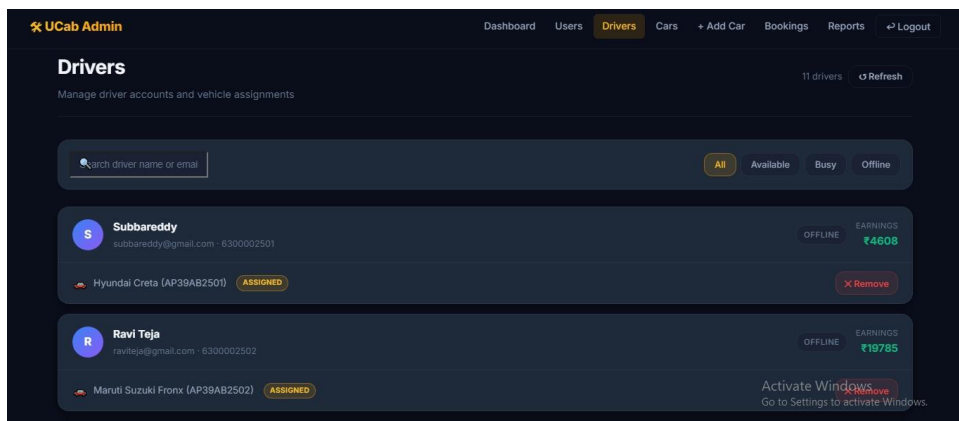


Figure 6: Admin drivers list with availability status and earnings

Rider — My Rides history with receipt download:

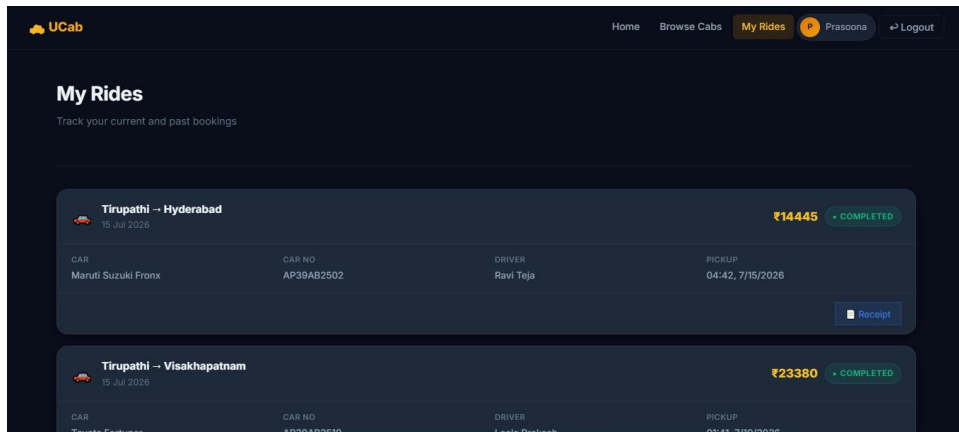


Figure 7: Rider ride history with completed trips and receipt access

Driver — Assigned rides detail view:

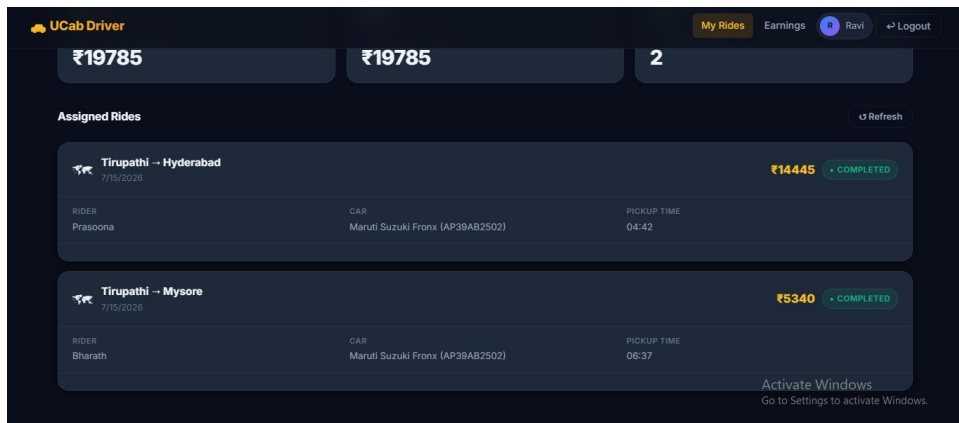


Figure 8: Driver's assigned rides with rider, car, and pickup-time details

GitHub Repository: <https://github.com/manikumar26092005/Cab-Booking-Premium>

Demo Video/Folder: <https://drive.google.com/drive/folders/1ZDpZ5FvcYlxTEkNpBHU6LG2rhIIjf5tR>

12. Known Issues

- Fare estimate could briefly show a stale value when the drop location was changed quickly (resolved by re-triggering fare calculation on location change).
- Concurrent bookings in the same area could occasionally attempt duplicate driver assignment (resolved by adding assignment locking at the database level).
- The application currently runs against a local MongoDB instance; multi-region or high-availability deployment has not yet been configured.

13. Future Enhancements

- Live map-based driver tracking during the 'On the Way' status

- In-app chat or call bridge between rider and driver
- Online payment gateway integration (currently fare is calculated and receipted, without a live payment gateway)
- Scheduled/advance ride booking refinements and reminders
- Migration from local MongoDB to MongoDB Atlas and cloud hosting for production scalability
- Push notifications for booking status changes